

**SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM  
CSÍKSZEREDAI KAR  
GAZDASÁGI INFORMATIKA SZAK**

**D I P L O M A D O L G O Z A T**

**Végzős hallgató:**

**Ördög Bernadett**

**Témavezető:**

**Dr. Pál László, egyetemi docens**

**2026**

**SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM  
CSÍKSZEREDAI KAR  
GAZDASÁGI INFORMATIKA SZAK**

**DIPLOMADOLGOZAT**

**MESTERSÉGES  
INTELLIGENCIA ALAPÚ  
KOCKÁZAT-ELŐREJELZÉS  
A PÉNZÜGYI PIACOKON**

**Végzős hallgató:**

**Ördög Bernadett**

**Témavezető:**

**Dr. Pál László, egyetemi docens**

**2026**

## Kivonat

A dolgozat a részvénytapi kockázatkezelés központi elemét, a volatilitás-előrejelzést vizsgálja az S&P 500 index példáján 2013–2025 között.

A kutatás fő célja annak megvizsgálása, hogy a modern neurális hálózaton alapuló modellek képesek-e felülmúlni a hagyományos ökonometriai modelleket a volatilitás-előrejelzésben. A vizsgált modellek: EWMA, GARCH, GJR-GARCH, HAR-RV, LSTM, valamint az LSTM-GARCH hibrid.

A modellek pontosságát a Garman-Klass realizált volatilitáshoz viszonyítva, Q-LIKE veszteségfüggvény és RMSE metrikák alapján értékeljük, gördülő ablakos (walk-forward) validációval, három eltérő volatilitás-rezsimben: stabil bikapiacon, COVID-sokk idején és kamatemelési periódusban.

A főbb eredmény, hogy mindkét piaci rezsimben az LSTM-alapú modellek teljesítettek a legjobban: stabil periódusban az LSTM-GARCH hibrid, extrém stresszperiódusban az önálló LSTM bizonyult a legpontosabbnak.

A Diebold–Mariano-tesztek alapján a teljesítménykülönbségek statisztikailag szignifikánsak, ami megerősíti, hogy a modellválasztás rezsimfüggő, és nincs egyetlen univerzálisan domináns megközelítés.

Kulcsszavak: volatilitás-előrejelzés, GARCH, LSTM, Garman-Klass, walk-forward validáció.

## Rezumat

Lucrarea analizează elementul central al gestionării riscului pe piața de acțiuni, și anume prognozarea volatilității, utilizând exemplul indicelui S&P 500 pentru perioada 2013–2025.

Obiectivul principal al cercetării este de a examina dacă modelele moderne bazate pe rețele neuronale sunt capabile să depășească modelele econometrice tradiționale în prognozarea volatilității. Modelele analizate sunt: EWMA, GARCH, GJR-GARCH, HAR-RV, LSTM, precum și hibridul LSTM-GARCH.

Acuratețea modelelor este evaluată în raport cu volatilitatea realizată Garman-Klass, pe baza funcției de pierdere Q-LIKE și a metricii RMSE, utilizând o validare prin fereastră rulantă (walk-forward validation), în trei regimuri diferite de volatilitate: o piață ascendentă (bull market) stabilă, perioada șocului COVID și perioada de majorare a ratelor dobânzii.

Principalul rezultat indică faptul că modelele bazate pe LSTM au performat cel mai bine în ambele regimuri de piață: în perioadele stabile, hibridul LSTM-GARCH s-a dovedit a fi cel mai precis, în timp ce în perioadele de stres extrem, modelul LSTM de sine stătător a oferit cele mai bune rezultate.

Pe baza testelor Diebold–Mariano, diferențele de performanță sunt semnificative din punct de vedere statistic, ceea ce confirmă faptul că alegerea modelului depinde de regimul pieței și că nu există o singură abordare universal dominantă.

Cuvinte-cheie: prognozarea volatilității, GARCH, LSTM, Garman-Klass, validare walk-forward.

## **Abstract**

The thesis examines volatility forecasting, a central element of equity market risk management, using the example of the S&P 500 index between 2013 and 2025.

The main objective of the research is to investigate whether modern neural network-based models can outperform traditional econometric models in volatility forecasting. The models examined are: EWMA, GARCH, GJR-GARCH, HAR-RV, LSTM, and the hybrid LSTM-GARCH.

The accuracy of the models is evaluated relative to the Garman-Klass realized volatility, based on the Q-LIKE loss function and RMSE metrics, using a rolling-window (walk-forward) validation across three distinct volatility regimes: a stable bull market, the COVID shock period, and the interest rate hike period.

The main finding is that LSTM-based models performed the best in both market regimes: the hybrid LSTM-GARCH proved to be the most accurate in stable periods, while the standalone LSTM was the most precise during extreme stress periods.

Based on the Diebold–Mariano tests, the performance differences are statistically significant, confirming that model selection is regime-dependent and that there is no single universally dominant approach.

Keywords: volatility forecasting, GARCH, LSTM, Garman-Klass, walk-forward validation.

## **A mesterséges intelligencia használata**

Alulírott Ördög Bernadett kijelentem, hogy a jelen „mesterséges intelligencia alapú kockázat előrejelzés a pénzügyi piacokon” elkészítése során generatív mesterséges intelligencia eszközt

- az alábbi módon vettem igénybe:

1. ChatGPT – helyesírás javítási céljából

2. Gemini – a témához kapcsolódó publikációk keresésének céljából használtam

Tudomásul veszem, hogy az MI által generált tartalomért teljes mértékben felelősséggel tartozom, és tisztában vagyok azzal, hogy az MI nem megfelelő vagy jelöletlen használata a Sapiaientia EMTE vonatkozó szabályzatai szerinti következményekkel járhat.

# Tartalomjegyzék

|                                                          |    |
|----------------------------------------------------------|----|
| 1. Bevezető.....                                         | 8  |
| 2. Szakirodalmi áttekintés .....                         | 9  |
| 3. Adatok és előfeldolgozás .....                        | 12 |
| 3.1. Adatforrás és időszak.....                          | 12 |
| 3.2. Piaci rezsimdek .....                               | 13 |
| 3.3. Leíró statisztika és feltáró elemzés (EDA) .....    | 13 |
| 3.4. ACF/PACF elemzés.....                               | 15 |
| 4. Módszertan .....                                      | 19 |
| 4.1. Benchmark: Garman-Klass realizált volatilitás ..... | 19 |
| 4.2. Klasszikus volatilitásmodellek.....                 | 20 |
| 4.2.1. EWMA .....                                        | 20 |
| 4.2.2. GARCH(1,1) .....                                  | 21 |
| 4.2.3. GJR-GARCH.....                                    | 22 |
| 4.2.4. HAR-RV .....                                      | 23 |
| 4.3. LSTM neurális hálózat .....                         | 24 |
| 4.3.1. Architektúra .....                                | 25 |
| 4.3.2. Kapumechanizmus.....                              | 26 |
| 4.3.3. Tanítás.....                                      | 27 |
| 4.4. LSTM-GARCH hibrid.....                              | 28 |
| 4.5. Gördülő ablakos validáció .....                     | 29 |
| 5. Értékelési keretrendszer .....                        | 31 |
| 5.1. Pontossági metrikák.....                            | 31 |
| 5.1.1. Q-LIKE veszteségfüggvény .....                    | 31 |
| 5.1.2. RMSE .....                                        | 32 |
| 5.2. Statisztikai szignifikancia .....                   | 32 |
| 5.2.1. Diebold–Mariano teszt.....                        | 33 |
| 5.2.2. Holm–Bonferroni korrekció.....                    | 34 |
| 6. Eredmények.....                                       | 35 |
| 6.1. Előrejelzési pontosság: Q-LIKE és RMSE.....         | 35 |
| 6.2. Volatilitás-előrejelzések vizualizációja .....      | 36 |

|                                            |           |
|--------------------------------------------|-----------|
| <b>6.3. Diebold–Mariano tesztek .....</b>  | <b>37</b> |
| <b>6.4. Rezsimfüggő teljesítmény .....</b> | <b>38</b> |
| <b>7. Következtetések .....</b>            | <b>40</b> |
| <b>Irodalomjegyzék .....</b>               | <b>41</b> |
| <b>Melléklet .....</b>                     | <b>43</b> |



## 1. Bevezető

A részvénytőzpiacok alapvető szerepet töltenek be a modern gazdaságban. A vállalatok finanszírozásának biztosítására és a befektetőknek a tőkéjük növelésére is szolgálhat. Ugyanakkor a forgalmukat eléggé befolyásolja a bizonytalanság, mivel az árfolyam folyamatosan változik. Így a kockázat mérése és előrejelzése kiemelt fontossággal bír a pénzügyi döntéshozatal során.

A volatilitás egy pénzügyi eszköz árának vagy hozamának ingadozása egy adott átlag körül (Poon, S. H., & Granger, 2003). A piac bizonytalanságát tükrözi. A volatilitás a részvénytőzpiaci kockázat egyik legfontosabb mérőszáma. A pontos előrejelzés segít meghatározni a potenciális veszteség mértékét.

A volatilitást évtizedekig Robert Engle által megalkotott ARCH (Robert F. Engle, 1982) és Tim Bollerslev által kidolgozott GARCH (Tim Bollerslev, 1986) modellek uralták. A gépi tanulás és a mély tanulás megjelenése új távlatokat nyitott meg, mivel összetettebb idősorokat is hatékonyan képes kezelni. Ennek következtében egyre népszerűbb a modern megközelítés, mint például az LSTM hálózat a hagyományos modellekkel szemben (Fischer, T., & Krauss, 2018).

Befektetés esetén a volatilitás-előrejelzés egy meghatározó mutató, amely segítségével a befektető meghatározza az eszközök közötti tőkeeloszlást. A befektetők elsődleges célja a hozam maximalizálása és a kockázat minimalizálása.

A kutatás fő célja annak a vizsgálata, hogy a modern neurális háló alapú modellek (LSTM) képesek-e felülmúlni a hagyományos ökonometriai modelleket volatilitás-előrejelzésben különböző piaci rezsimekben.

A dolgozatban vizsgált kérdések: Mennyivel jobb az LSTM vagy hibrid LSTM-GARCH megközelítés a standard modellekhez képest? Melyik modell teljesít a legjobban stressz-periódusban (COVID-sokk) vagy stabil rezsimben?

A dolgozat az alábbiak szerint épül fel: a 2. fejezet a releváns szakirodalmat tekinti át, a 3. fejezet az adatokat és azok előfeldolgozását mutatja be, a 4. fejezet a módszertant ismerteti, az 5. fejezet az értékelési keretrendszert, a 6. fejezet az eredményeket tárgyalja, a 7. fejezet pedig a következtetéseket foglalja össze. A dolgozat az irodalomjegyzékkel és a melléklettel zárul.

A programkód a következő címen érhető el github-on: <https://github.com/OrdogBernadett/Allamvizsga.git>.

## 2. Szakirodalmi áttekintés

A pénzügyi volatilitás előrejelzésére az évtizedek során számos modell született, az egyszerű ökonometriai megközelítésekől a mély tanulási architektúrákig.

Az ARCH (Autoregressive Conditional Heteroscedasticity) modellt Robert Engle alkotta meg 1982-ben a pénzügyi idősorok időben változó volatilitásának modellezésére (Engle, 1982). Engle munkásságáért 2003-ban közgazdasági Nobel-díjat kapott.

Bár az ARCH modell úttörő volt, a megfelelő illeszkedés sok kísérletezést igényelt. Ezért Tim Bollerslev 1986-ban kifejlesztette a GARCH (Generalized Autoregressive Conditional Heteroscedasticity) modellt, amely az ARCH kiterjesztéseként a múltbeli varianciát is beépíti a feltételes szórásbecslésbe (Bollerslev, 1986).

A GJR-GARCH (Glosten–Jagannathan–Runkle GARCH) modellt Lawrence R. Glosten, Ravi Jagannathan és David E. Runkle fejlesztette ki 1993-ban a GARCH aszimmetriakezelési korlátainak kiküszöbölésére: a modell képes megkülönböztetni a pozitív és negatív hozamsokkok eltérő volatilitáshatását (Glosten et al., 1993).

Az EGARCH (Exponential GARCH) modellt Daniel Nelson dolgozta ki 1991-ben. A GARCH korlátaival szemben az EGARCH exponenciális specifikációja nemnegatív paraméter-megkötések nélkül képes modellezni a tőkeáttételi hatást (Nelson, 1991).

Fulvio Corsi 2009-ben fejlesztette ki a HAR-RV (Heterogeneous Autoregressive model of Realized Volatility) modellt, amelynek célja a pénzügyi idősorokban megfigyelhető hosszú memória jelenségének megragadása és a különböző időhorizontok hatásának egyidejű kezelése (Corsi, 2009).

A mesterséges neurális hálózatok megjelenése a pénzügyi idősorok elemzésében alapvető változásokat hozott, mivel ezek a módszerek bonyolult, nemlineáris összefüggéseket is képesek kezelni. Az áttörést a rekurens neurális hálózat (RNN) jelentette, amelyet kifejezetten szekvenciális adatok feldolgozására fejlesztettek ki. Egyik legfontosabb újítása, hogy az RNN architektúra rendelkezik memóriával, így képes megjegyezni a korábban feldolgozott értékeket.

A Long Short-Term Memory (LSTM) architektúrát Sepp Hochreiter és Jürgen Schmidhuber fejlesztette ki 1997-ben azzal a céllal, hogy leküzdjék az RNN legsúlyosabb korlátját, a gradiensejtűnés problémáját (Hochreiter & Schmidhuber, 1997). Az architektúrát a *Neural Computation* című szaklapban ismertették; azóta széles körben alkalmazzák jövőbeli záróárak és hozamirányok becslésére.

Tanulmányok igazolták, hogy az LSTM architektúra pontosabb eredményeket nyújt, mint a memória nélküli modellek (Fischer & Krauss, 2018; Bucci, 2020). Fischer és Krauss 2018-as

munkája – „Deep learning with long short-term memory networks for financial market predictions” (*European Journal of Operational Research*) – az S&P 500-re vizsgálta az LSTM előrejelző képességét, és igazolta, hogy a modell képes automatikusan megtanulni a részvényárak történelmi mintázatait.

Andrea Bucci 2020-ban megjelent „Realized volatility forecasting with neural networks” című tanulmánya szintén megerősítette, hogy a neurális hálózatok szignifikánsan pontosabb előrejelzéseket adnak, mint a klasszikus statisztikai modellek (Bucci, 2020).

Kim és Won 2018-ban publikálták a „Forecasting the volatility of stock price index: A hybrid model integrating LSTM with multiple GARCH-type models” tanulmányukat az *Expert Systems with Applications* folyóiratban. A szerzők olyan hibrid architektúrát dolgoztak ki, amely ötvözi az LSTM és a GARCH modell előnyeit a részvényindexek volatilitásának pontosabb előrejelzése érdekében; az eredmények szerint a hibrid megközelítés szignifikánsan felülmúlta az önálló modelleket (Kim & Won, 2018).

A Nobel-díjas Clive Granger és Ser-Huang Poon 2003-as „Forecasting volatility in financial markets: A review” összefoglalójukban 93 korábbi kutatást vizsgáltak meg. Következtetésük szerint a bonyolultabb ARCH- vagy GARCH-modellek nem egyértelműen jobbak az egyszerűbb megközelítésekénél; az előrejelzés sikerét három tényező határozza meg leginkább: az adatok gyakorisága, a viszonyítási alap megválasztása és az előrejelzési időtáv (Poon & Granger, 2003).

Andrew Patton munkássága alapvető a volatilitás-előrejelzések értékelésében, különösen a Q-LIKE (Quasi-Likelihood) veszteségfüggvény alkalmazása és a proxy-robosztusság elméleti megalapozása terén. Mivel a volatilitás közvetlenül nem megfigyelhető, szükség van egy viszonyítási alapra (proxy), amelyhez az előrejelzések mérhetők. A Q-LIKE egy robusztus veszteségfüggvény, amelyet elsősorban volatilitás-előrejelző modellek teljesítményének értékelésére és rangsorolására használnak (Patton, 2011).

Diebold és Mariano 1995-ben javasolt tesztje két versengő előrejelző modell pontosságát hasonlítja össze statisztikai alapon. A tesztet széles körben alkalmazzák a volatilitás-előrejelzési modellek összehasonlítására, köztük az S&P 500 indexre vonatkozó tanulmányokban is (Diebold & Mariano, 1995).

Magyar tőkepiaci kontextusban Gál (2022) a GJR-GARCH és EGARCH modellek összehasonlításával vizsgálta az OTP és MOL részvények volatilitásának előrejelzését. Az empirikus elemzés igazolta, hogy az aszimmetrikus GARCH-modellek statisztikailag

pontosabb előrejelzést nyújtanak az alap GARCH(1,1) modellnél, és a nagy gazdasági válságok idején jelentősebb volatilitás figyelhető meg a részvényárfolyamok hozamaiban.

### 3. Adatok és előfeldolgozás

A következő fejezetben az elemzéshez felhasznált adatokat és az előfeldolgozás lépéseit mutatjuk be. A kutatás megbízhatóságához elengedhetetlen a releváns adatok gondos feldolgozása.

#### 3.1. Adatforrás és időszak

Az elemzés alapját az S&P 500 index (^GSPC) napi OHLCV adatsor képezi, amelyet a Yahoo Finance API-n keresztül töltöttünk le. Az adatsor a 2013.01.01 – 2025.01.01 időszakot foglalja magába, ami összességében 3019 kereskedési napnak felel meg (lásd a 3.1. ábrát). A számítások Python 3.10.19 környezetben készültek, a felhasznált csomagokat az 1. melléklet tartalmazza. A *TICKER* a ^GSPC ez az index Yahoo Finance szimbóluma. A *START\_DATE* és az *END\_DATE* a kezdő és a végső dátum. A *TDPY* a kereskedési napokat jelöli, ami egy évben 252 nap, ez a standard elfogadott érték az egy évre jutó kereskedési napok számának (hétvégék és tőzsdei ünnepek levonása után).

A napi log-hozamok időben összehasonlíthatók és szimmetrikusak, így lehetővé teszik a különböző időtávú hozamok egyszerű matematikai kezelését és az eszközök összehasonlítását. Kiszámításukkal a trendeket követő ársorok stacionáriussá alakíthatóak, ami alapfeltétele az olyan összetett modellek alkalmazásának, mint a GARCH vagy az LSTM.

A napi log-hozamot a következőképpen számíthatjuk:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right), \quad (1)$$

ahol  $P_t$  a  $t$ . nap záróárfolyama.

```
TICKER      = '^GSPC'  # S&P 500 index Yahoo Finance szimbóluma
START_DATE  = '2013-01-01'
END_DATE    = '2025-01-01'
ROLLING_WINDOWS = [1000] # 1000 napos ablak
TDPY        = 252 # kereskedési napok évente
```

3.1. ábra. Az empirikus elemzés kiinduló paraméterei

### 3.2. Piaci rezsimek

Három jól elkülönülő periódust vizsgálunk (lásd a 3.1. táblázatot). A három periódus különböző volatilitás-rezsimeket képvisel. Ez lehetővé teszi annak vizsgálatát, hogy az egyes modellek mennyire robusztusak eltérő piaci körülmények között.

A bikapiac a pénzügyi eszközök tartós emelkedésével jellemezhető időszak, amelyben a piaci környezet kedvező feltételeket biztosít a befektetések értékének növekedéséhez – ezt képviseli a Pre-COVID-periódus alacsony, 12,3%-os volatilitással. A stresszperiódus extrém piaci sokkot jelent: a COVID-járvány 2020-as kitörésekor a volatilitás 36,1%-ra emelkedett, jóval meghaladva a normál szintet. Ezzel szemben a medvepiac időszakában az árfolyamok tartósan esnek, ami komoly veszteségkockázatot jelent a piaci szereplők számára – ezt a 2022-es kamatemelési periódus reprezentálja 24,2%-os volatilitással.

| Periódusok  | Dátum                   | Napok | Volatilitás | Jelleg                  |
|-------------|-------------------------|-------|-------------|-------------------------|
| Pre-COVID   | 2019-01-01 – 2020-01-31 | 272   | 12,3%       | Alacsony vol., bikapiac |
| COVID-sokk  | 2020-02-01 – 2020-12-31 | 231   | 36,1%       | Extrém stressz          |
| Kamatemelés | 2022-01-01 – 2022-10-21 | 251   | 24,2%       | Tartós medvepiac        |

3.1. táblázat. Elemzett piaci periódusok

### 3.3. Leíró statisztika és feltáró elemzés (EDA)

A leíró statisztikák (átlag, szórás, ferdeség, csúcsosság) a vizsgált idősorok alapvető tulajdonságainak jellemzésére szolgálnak, és lehetővé teszik a különböző piaci periódusok közötti összehasonlítást. A mutatók és a normál eloszlásvizsgálathoz használt Jarque-Bera teszt programozási megvalósítását a 3.3. ábra szemlélteti. A kód első sorában, a *mu*, *sigma*, parancs a hozamok adatsorának átlagát és szórását határozza meg a Pandas könyvtár beépített matematikai funkcióival. A szórás a pénzügyi elemzésekben a volatilitás, vagyis a kockázat elsődleges mérőszáma. A harmadik és negyedik sorban a *scipy.stats* csomag függvényei segítségével az eloszlás alakját írjuk le. A *stats.skew(r)* a ferdeséget számítja ki. a ferdeséget számítja ki. A negatív érték a balra elnyúló eloszlást, vagyis a hirtelen, nagy árfolyamesések magasabb gyakoriságát jelzi. A *stats.kurtosis(r)* parancs a többlet-csúcsosságot adja meg. Végül a *jarque\_bera(r)* függvény végzi el a formális hipotézisvizsgálatot, amely a ferdeség és

a csúcsosság együttes mintázata alapján ellenőrzi a normál eloszláshoz való illeszkedést. A függvény két értéket ad vissza: a tesztstatisztikát ( $jb\_stat$ ) és a hozzá tartozó p-értéket ( $jb\_p$ ).

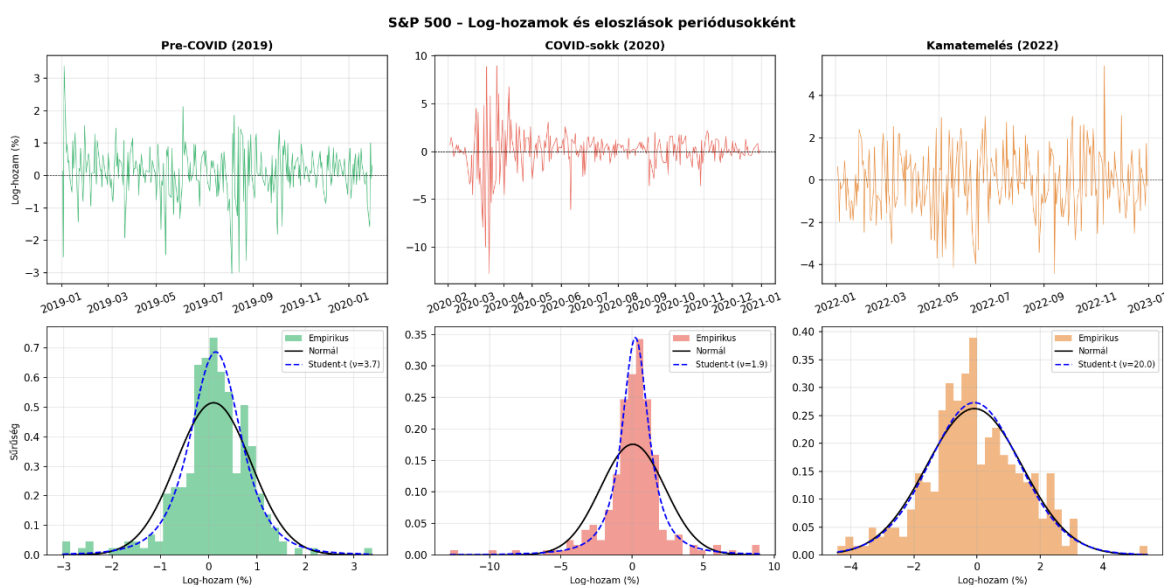
A Jarque-Bera teszt azt ellenőrzi, hogy az idősor ferdesége és csúcsossága illeszkedik-e a normál eloszláshoz.

Ahogy az a 3.2. táblázatban is látható, a szórás a COVID-sokk periódusában közel háromszorosa a Pre-COVID-értéknek. A ferdeség mindhárom periódusban negatív, ami azt jelzi, hogy a nagy esések gyakoribbak, mint a nagy emelkedések. A Jarque-Bera teszt eredménye azt mutatja, hogy az első két periódusban az eloszlás szignifikánsan eltér a normálistól ( $p = 0,000$ ), míg a kamatemelési periódusban ez nem igazolható ( $p = 0,553$ ).

| Periódus    | Átlag (Mean) | Szórás (Std) | Ferdeség (Skew) | Csúcsosság (Kurt) | Jarque-Bera teszt |
|-------------|--------------|--------------|-----------------|-------------------|-------------------|
| Pre-COVID   | +0.099%      | 0.755%       | -0.65           | 3.21              | 0.000             |
| COVID-sokk  | +0.063%      | 2.271%       | -0.84           | 7.72              | 0.000             |
| Kamatemelés | -0.086%      | 1.521%       | -0.01           | 0.34              | 0.553             |

3.2. táblázat. Leíró statisztika értékei

A Student-t eloszlás a normál eloszláshoz képest „vastagabb farokkal” rendelkezik, így nagyobb valószínűséget rendel a szélsőséges eseményekhez. Ezért sokkal élethűbben tükrözi a tőzsdei válságokat, mint a hagyományos haranggörbe (Bollerslev, 1987). Ezt mutatja a 3.2. ábra is, ahol jól látható, hogy a Student-t jobban illeszkedik az empirikus eloszláshoz, mint a normál eloszlás, különösen a COVID-sokk időszakában.



3.2. ábra. Log-hozamok és eloszlások

```
mu, sigma = r.mean(), r.std()
skew = stats.skew(r); # ferdeség: negatív (nagy esések gyakoribbak)
kurt = stats.kurtosis(r) # csúcsosság: >0 → leptokurtikus, vastag farkú eloszlás
jb_stat, jb_p = jarque_bera(r) # Jarque-Bera teszt:  $H_0$ : normális eloszlás -  $p < 0.05 \rightarrow$  elveti
```

**3.3. ábra.** A leíró statisztikai mutatók kiszámításának forráskódja

### 3.4. ACF/PACF elemzés

Az ACF (autokorrelációs függvény) azt méri, hogy egy idősorban mekkora az összefüggés a saját korábbi értékeivel, míg a PACF (parciális autokorrelációs függvény) ugyanezt méri azzal a kiegészítéssel, hogy kiszűri a köztes tagok közvetett hatásait. Segítségükkel eldönthető, hogy egy folyamat korábbi hibákból vagy korábbi értékekből táplálkozik-e, így pontosabbá válik a jövőbeli volatilitás becslése is.

A számítások és a grafikus megjelenítés mögötti programozási logikát a 3.7. ábra szemlélteti, amely egy 2x2 rendszerezi az eredményeket, egységesen 30 napi késleltetést ( $\text{lags}=30$ ) és 5%-os szignifikanciaszintet ( $\alpha=0.05$ ) alkalmazva.

A kód első blokkja a nyers log-hozamok lineáris kapcsolatát vizsgálja a *plot\_acf* és *plot\_pacf* függvények segítségével. A második blokk a négyzetes hozamok autokorrelációját térképezi fel. Mivel a hozamok négyzetre emelésével kiküszöböljük az előjeleket, ez a transzformáció a hozamok nagyságrendjét (a varianciát) ragadja meg, így közvetlenül alkalmas a tőkepiacokra jellemző volatilitás-klaszterezés kimutatására. Végül a harmadik egység az abszolút hozamok autokorrelációját ábrázolja, amely a négyzetes hozamoknál robusztusabb volatilitás-proxyként működik, mivel kevésbé érzékeny a mintában előforduló extrém kiugró értékek torzító hatásaira.

Az így generált diagramok segítségével vizsgált alperiódusok eredményeit a 3.4., 3.5. és 3.6. ábrák foglalják össze. Az ábrákon kiolvasható, hogy a Pre-COVID periódusban a nyers hozamoknál nincs szignifikáns autokorreláció, ami azt mutatja, hogy a múltbeli hozamból nem jósolható a jövőbeli hozam. Ezzel szemben a négyzetes és abszolút hozamok autokorrelációja  $\text{lag}=1$  körül szignifikáns, majd lassan lecsengő. Ez mérsékelt volatilitás-klaszterezésre utal, ami indokolja az egyszerűbb GARCH(1,1) modell alkalmazását.

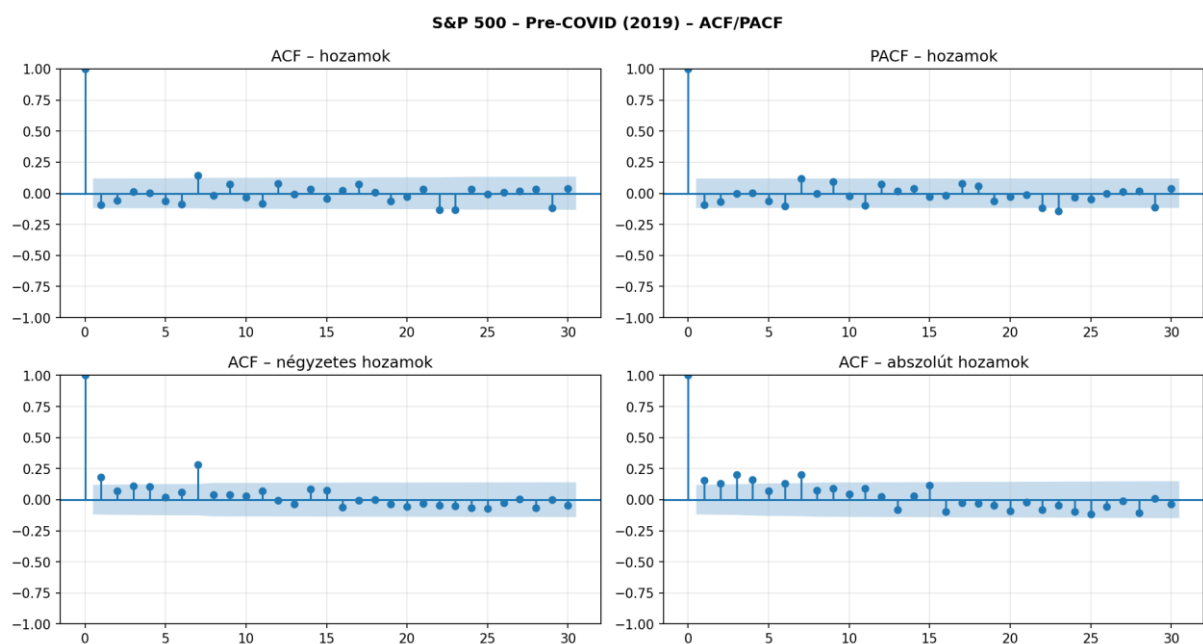
A COVID-sokk periódusában a nyers hozamok autokorrelációja  $\text{lag}=1-2$  körül szignifikáns, ami arra utal, hogy a piac hatékonysága átmenetileg sérült a pánikkereskedés miatt. A négyzetes hozamok autokorrelációja  $\text{lag}=10$ -ig szinte végig szignifikáns, erős, tartós



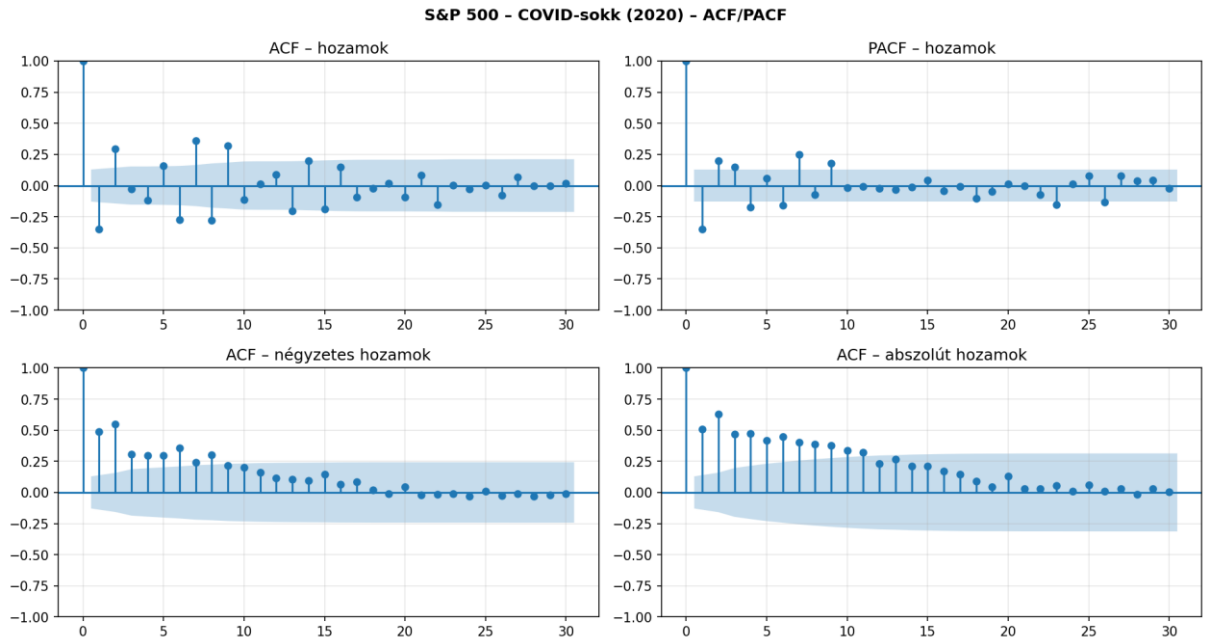
volatilitás-perzisztenciát jelezve. Ez indokolja az LSTM előnyét, amelynek 60 napos ablaka ezt a hosszú memóriát megragadja.

A kamatemelési ciklus előre bejelentett, fokozatos folyamat volt, ezért a kamatemelési periódusban minden panel szinte teljesen a szignifikanciasávon belül mozgott. A modellek közötti különbség így kisebb, az előrejelzés pedig nehezebb, mivel nincs kihasználható szerkezet.

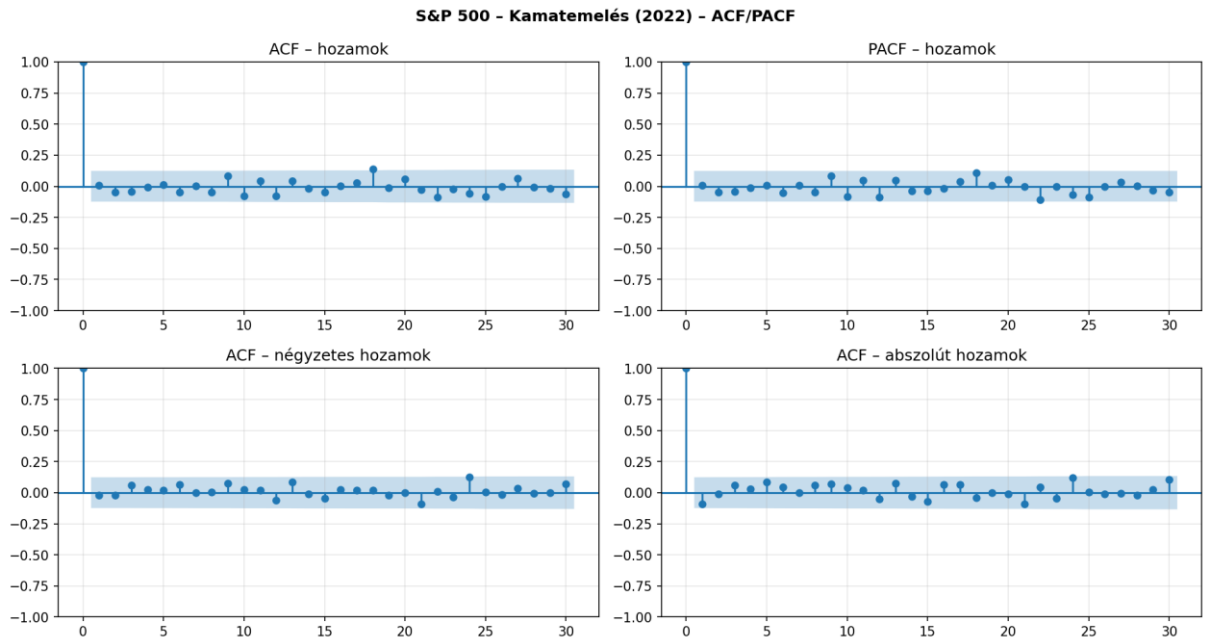
Az ACF-elemzés megerősíti, hogy a három periódus ténylegesen különböző volatilitás-rezsimeket képvisel, és indokolja a modell-összehasonlítás elvégzését mindhárom időszakban.



**3.4. ábra.** Pre-COVID ACF/PACF elemzések



**3.5. ábra.** COVID-sokk ACF/PACF elemzések



**3.6. ábra.** Kamatemelés ACF/PACF elemzések

```

# nyers hozamok autokorrelációja: ARMA struktúra vizsgálata
plot_acf( r, ax=axes[0,0], lags=30, alpha=0.05, title='ACF - hozamok')
plot_pacf(r, ax=axes[0,1], lags=30, alpha=0.05, title='PACF - hozamok')
# négyzetes hozamok ACF-je: volatilitás-klaszterezés kimutatása
plot_acf( r**2, ax=axes[1,0], lags=30, alpha=0.05, title='ACF - négyzetes hozamok')
# abszolút hozamok ACF-je: robusztusabb volatilitás-proxy
plot_acf( r.abs(), ax=axes[1,1], lags=30, alpha=0.05, title='ACF - abszolút hozamok')

```

**3.7. ábra.** A volatilitás-klaszterezés és emlékezet vizsgálatának forráskódja.

## 4. Módszertan

### 4.1. Benchmark: Garman-Klass realizált volatilitás

A modell teljesítményét egy realizált volatilitás-proxy segítségével mérjük. Az irodalomban több OHLC-alapú becslő is elterjedt; ezek összehasonlítása alapján a Garman–Klass-becslőt választottuk. A számítási lépések algoritmikus megvalósítása az 4.1. ábrán található. A hagyományos becslők csak a nyitó és záróárfolyamokat veszik figyelembe, míg a GK a napközbeni árfolyammozgásokat is felhasználja, így pontosabb képet ad a piaci feszültségről:

$$\sigma_{GK,t}^2 = \frac{1}{2} \left( \ln \frac{H_t}{L_t} \right)^2 - (2 \ln 2 - 1) \left( \ln \frac{C_t}{O_t} \right)^2 \quad (2)$$

ahol  $H_t$ ,  $L_t$ ,  $O_t$ ,  $C_t$  a napi csúcs, völgy, nyitó és záróárfolyam (Garman & Klass, 1980). A napi értékből éves volatilitást kapunk.

A kód szintjén a `compute_garman_klass_rv` függvény először elkészíti a modell két alapelemét: a `log_hl` változó a napi maximum és minimum árfolyamok logaritmus aránya, míg a `log_co` a záró- és nyitóárak logaritmus arányát kalkulálja ki. Ezt követően a `gk_sq` sorban az előáll a napi varianciasor.

Mivel a napi adatok rendkívül zajosak, a függvény a return ágban egy 21 napos mozgóablakot alkalmazva simítja az adatokat a `.rolling().mean()` parancslánccal, miközben a `min_periods=window//2` paraméter biztosítja a robusztusságot az esetlegesen hiányzó adatokkal szemben. A negatív varianciaértékek matematikai kizárására a `clip(lower=0)` metódus szolgál. Végül a kapott értékből a négyzetgyökvonást végzünk, amelyet a kereskedési napok elméleti számával való szorzással évesített szintű realizált volatilitássá transzformálunk.

```
def compute_garman_klass_rv(df, window=21, tdp=252):
    # napi magas-alacsony log-arány (Garman-Klass képlet első tagja)
    log_hl = np.log(df['High'] / df['Low'])
    # napi záró-nyitó log-arány (Garman-Klass képlet második tagja)
    log_co = np.log(df['Close'] / df['Open'])
    # Garman-Klass variance becslő:  $\sigma^2 \approx 0.5 \cdot (H/L)^2 - (2 \ln 2 - 1) \cdot (C/O)^2$ 
    gk_sq = 0.5 * log_hl**2 - (2 * np.log(2) - 1) * log_co**2
    # rolling átlag → simított, évesített GK volatilitás ( $\sigma \cdot \sqrt{252}$ )
    return np.sqrt(gk_sq.rolling(window, min_periods=window//2).mean()).clip(lower=0) * tdp
```

4.1. ábra. Rolling RV (benchmark)

## 4.2. Klasszikus volatilitásmodellek

### 4.2.1. EWMA

Az EWMA egy olyan mozgóátlag, amely az időben közelebbi adatoknak nagyobb súlyt ad, míg a régebbiekét fokozatosan elfelejti. Ez a modell rekurzívan frissíti a varianciabecslést:

$$\hat{\sigma}_t^2 = \lambda \hat{\sigma}_{t-1}^2 + (1 - \lambda) r_{t-1}^2, \quad \lambda = 0,94 \quad (3)$$

A  $\lambda=0,94$  értéke a JP Morgan RiskMetrics kutatóinak köszönhető, akik az 1990-es években több ezer pénzügyi adatsort teszteltek (RiskMetrics, 1996). A választás egyik fő indoka, hogy ha ennél az értéknél nagyobb lenne, akkor a modell túl lassan reagálna az aktuális piaci sokkokra; ha viszont kisebb lenne, akkor a volatilitás-becslés minden kisebb eseményre túlérzékenyen ingadozna. A modell célja a piacon megfigyelt volatilitás-tömörülés kezelése, ahol a nagy árváltozásokat nagyok, a kicsiket kicsik követik. A 0,94-es érték lehetővé teszi, hogy a becslés kellően gyorsan alkalmazkodjon a megváltozott piaci hangulathoz, miközben megtartja a statisztikai stabilitást.

A konkrét programozási megvalósítás a Pandas könyvtár exponenciális ablakkezelő metódusára épül. Az `ewma_val = train.ewm(alpha=1 - lambda_ewma).std()` parancslánc a hozamok tanító adatsorára illeszti a modellt. Fontos technikai részlet, hogy a Pandas `.ewm()` függvénye a simítási tényezőt nem a lambda, hanem az alpha paraméteren keresztül várja. A `.std()` függvényhívás közvetlenül a volatilitást származtatja a varianciából. Mivel a mozgóablak a teljes idősorra legenerálja a becsléseket, az `.iloc[-1]` indexeléssel szűrjük ki a legfrissebb napi értéket. Ezt a napi szintű volatilitást végül a `np.sqrt(252)` kifejezéssel szorozva transzformáljuk évesített értékre.

Az EWMA formálisan az IGARCH(1,1) speciális esetének tekinthető, ahol  $\alpha+\beta=1$ . Nem tartalmaz hosszú távú átlaghoz való visszatérést ( $\omega=0$ ), ezért reaktívabb, mint a GARCH – rövid előrejelzési horizonton ez előny lehet.

```
LAMBDA_EWMA = 0.94 # RiskMetrics
# EWMA std: exponenciálisan súlyozott mozgóátlag, λ=0.94
ewma_val = train.ewm(alpha=1 - lambda_ewma).std().iloc[-1] * np.sqrt(252)
```

4.2. ábra. Az EWMA volatilitásbecslés paraméterezése

#### 4.2.2. GARCH(1,1)

Bollerslev (1986) cikke azért vált a modern pénzügyi ökonometria alapkövévé, mert az ARCH modellt kiegészítette a múltbeli varianciaértékeivel, így megalkotva az általánosabb és pontosabb GARCH modellt. Az újítás lehetővé tette, hogy kevesebb paraméterrel is pontosabb eredmény érhető el, mivel a modell képes megragadni a tőzsdén megfigyelhető volatilitás-perzisztenciát:

$$\begin{aligned} r_t &= \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t z_t, \quad z_t \sim t_v(0,1), \\ \sigma_t^2 &= \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2 \end{aligned} \quad (4)$$

Ahol:

- $\mu$  – a hozam várható értéke
- $\omega$  – konstans a varianciaegyenletben
- $\alpha$  – múltbeli sokkok hatása
- $\beta$  – múltbeli volatilitás hatása
- $v$  – a Student-eloszlás szabadságfoka

A modell algoritmikus implementációját és az előrejelzés menetét a 4.3. ábra mutatja be. A modell paramétereinek a meghatározására az MLE (Maximum Likelihood Estimation) eljárást használják, ami megkeresi azokat az értékeket, amelyek mellett a megfigyelt adatsor a lehető legnagyobb valószínűséggel következik be.

Technikai szempontból fontos módszertani lépés, hogy az `arch_model` függvény a stabilabb numerikus konvergencia érdekében a százalékos formátumú hozamokat preferálja. Emiért a kód a bemenő adatokat százzal szorozza. A modell illesztése után a `m.forecast(horizon=1).variance` parancs legenerálja az egy lépéssel előretekintő feltételes variancia-előrejelzést. Mivel a számítás végig százalékos skálán, a return ágban a kapott értéket először négyzetgyök alá vonjuk, majd visszaskálázzuk eredeti arányszámmá. Végül az EWMA-hoz hasonlóan a tőzsdei kereskedési napok számával, azaz a `np.sqrt(252)` értékkel való szorzással kapjuk meg a végső, évesített szintű GARCH volatilitást.

A Student-t eloszlás alkalmazása a normálisnál vastagabb farkú hozameloszlást kezeli hatékonyabban (Bollerslev, 1987). Stacionárius feltétele  $\alpha + \beta < 1$ , amely biztosítja, hogy a volatilitás ne szálljon el a végtelenbe, hanem a sokk után mindig visszatérjen a hosszú távú átlaghoz.

```

garch_params = arch_model(
    train100, vol='Garch', p=1, q=1, dist='t'
).fit(dis='off', show_warning=False).params

# 1 napos előrejelzés
m = arch_model(train * 100, vol=vol, p=1, o=0, q=1, dist='t').fix(params)
# egy napra előre kondicionális variancia előrejelzés
v = m.forecast(horizon=1).variance.values[-1, 0]
# visszaskálázás: %-ból arányba (/100), napi→éves (x√252)
return np.sqrt(v) / 100 * np.sqrt(252) |

```

4.3. ábra. A GARCH(1,1) modell paraméterbecslése

### 4.2.3. GJR-GARCH

Glosten et al. (1993) aszimmetrikus modellbővítése a tőkepiaci tőkeáttétel-effektust (leverage-effect) ragadja meg:

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \gamma \varepsilon_{t-1}^2 1[\varepsilon_{t-1} < 0] + \beta \sigma_{t-1}^2 \quad (5)$$

Ahol:

- $\sigma_t^2$  – feltételes variancia
- $\omega$  – konstans alapszint
- $\alpha \varepsilon_{t-1}^2$  – múltbeli sokk hatása (ARCH hatása)
- $\beta \sigma_{t-1}^2$  – volatilitás perzisztenciája (GARCH hatás)
- $\gamma \varepsilon_{t-1}^2 1[\varepsilon_{t-1} < 0]$  – extra hatás, ha a sokk negatív;  $1[\varepsilon_{t-1} < 0]$  – indikátorfüggvény (1 ha igaz, különben 0)

Ahhoz, hogy a modell által becsült variancia mindig pozitív maradjon, szigorú korlátoknak kell teljesülniük:  $\omega > 0, \alpha \geq 0, \beta \geq 0$  és  $(\alpha + \gamma) \geq 0$ .

A modell algoritmikus megvalósítását és az aszimmetrikus struktúra specifikációját a 4.4. ábra mutatja be. A kód a `vol='Garch'` deklaráció mellett a  $p=1$  és  $q=1$  paraméterekkel határozza meg az ARCH és GARCH tagok késleltetését, miközben a `dist='t'` opcióval biztosítja, hogy a becslési eljárás a korábban azonosított csúcsosabb valószínűségi eloszlás tulajdonságok miatt a vastag farkú Student-t eloszlást kövesse. A `fit(dis='off')` metódus elrejt a számítási lépések részleteit, és közvetlenül a becsült paramétervektort (`.params`) adja vissza.

Az aszimmetrikus leverage egy olyan effektus, amely során a negatív hozamok ( $\gamma < 0$ ) sokkal nagyobb volatilitás-növekedést idéznek elő, szemben egy hasonló pozitív hozammal.

Az S&P 500 indexnél a kutatások igazolták az aszimmetrikus leverage-hatást, mivel a negatív sokk jelentősen növeli a jövőbeli volatilitást, mint az azonos hatású pozitív piaci hatások.

```
gjr_params = arch_model(  
    # GJR-GARCH: o=1 az aszimmetrikus Leverage-tag  
    train100, vol='Garch', p=1, o=1, q=1, dist='t'  
) .fit(dispatch='off', show_warning=False).params
```

4.4. ábra. A GJR-GARCH(1,1) modell leverage-paraméterének becslése

#### 4.2.4. HAR-RV

Corsi (2009) heterogén autoregresszív realizált volatilitás modellje napi, heti és havi komponenseket kombinál:

$$\ln(GK_t) = c + \beta_d \ln(GK_{t-1}^{(d)}) + \beta_\omega \ln(GK_{t-1}^{(\omega)}) + \beta_m \ln(GK_{t-1}^{(m)}) + \varepsilon_t \quad (6)$$

Ahol  $GK^{(d)}$ ,  $GK^{(\omega)}$ ,  $GK^{(m)}$  a napi, 5 napos, illetve a 22 napos átlagos GK értékek egy napos eltolással a look-ahead bias elkerülése érdekében. A look-ahead bias egy olyan hiba, ami akkor keletkezik, mikor az előrejelzésben olyan információkat használ fel, amely a valóságban még nem áll a rendelkezésére. Ezért ennek elkerülése elengedhetetlen, mivel a torzítás irreálisan feljavíthatja a modell teljesítményét.

A kód ezt követően építi fel a (6)-os egyenlet három független változóját. A napi komponenst `df['log_GK_d']` a `gk_lag` közvetlen logaritmizálásával kapjuk meg, ahol a `.clip(lower=eps)` függvény biztosítja, hogy a volatilitás ne csökkenjen a gép által még pontosan kezelhető legkisebb érték alá, elkerülve a nullával való osztást vagy a negatív végtelen logaritmusokat. A heti komponenst `df['log_GK_w']` egy 5 napos gördülő ablak átlagolásával, a `.rolling(5).mean()` metódusok alkalmazásával, míg a havi komponenst `df['log_GK_m']` egy 22 napos tőzsdei munkahónapnak megfelelő gördülő ablak átlagolásával, a `.rolling(22).mean()` metódusok segítségével származtatja a kód, mindkét esetben alkalmazva a log-transzformációt.

A változók előállítás után az együtthatók becslése a standard legkisebb négyzetek módszerével (OLS) történik. Végül a jövőbeli volatilitás becslése a `vol_pred = np.exp(...)` sorral valósul meg. Mivel a lineáris regresszió a log-volatilitás skáláján tesz előrejelzést, a kapott eredményen végre kell hajtani az inverz transzformációt.



A heterogén befektetői horizont koncepciója arra épül, hogy a piaci szereplők eltérő időtávokon (napi, havi, évi) kereskednek, így különbözőképpen észlelik a kockázatokat és reagálnak a piaci sokkokra.

```
# shift(1): leakage-mentes, az aznapi adat nem kerül be a featurekbe
gk_lag = df['GK_daily'].shift(1)
# napi HAR-RV komponens (log-skála): tegnapi volatilitás
df['log_GK_d'] = np.log(gk_lag.clip(lower=eps))
# heti HAR-RV komponens: 5 napos visszatekintő átlag
df['log_GK_w'] = np.log(gk_lag.rolling(5).mean().clip(lower=eps))
# havi HAR-RV komponens: 22 napos visszatekintő átlag
df['log_GK_m'] = np.log(gk_lag.rolling(22).mean().clip(lower=eps))

# OLS illesztés:
har_model = LinearRegression().fit(X_har, y_har)

# Előrejelzés visszatranszformálással:
vol_pred = np.exp(har_model.predict(X_t.reshape(1, -1))[0])
```

4.5. ábra. A log-HAR-RV modell napi, heti és havi komponenseinek előállítás

### 4.3. LSTM neurális hálózat

A hagyományos ökonometriai modellekkel (GARCH, HAR-RV) szemben a mély tanulási (deep learning) algoritmusok képesek megragadni a tőkepiaci idősorokban rejlő komplex, nemlineáris összefüggéseket és a rendkívül hosszú távú memóriahatásokat. A kutatás során alkalmazott neurális hálózati architektúra specifikációját és algoritmikus felépítését a 4.6. ábra szemlélteti.

Az architektúra strukturális felépítése a hálózat a folyamat elején, az `__init__` függvényben történik. A modell magját egy beépített, kétrétegű LSTM struktúra alkotja. A hálózat bemeneti dimenziója `input_size=1`, ami azt jelenti, hogy a modell lépésenként, egyszerre csak egy értéket dolgoz fel.

A túliilleszkedés hatékony elkerülése érdekében az LSTM rétegek közé egy 10%-os lemorzsolódási ráta került beépítésre. Fontos technikai részlet, hogy a kód egy logikai feltétellel, `dropout if num_layers > 1 else 0.0`, kezeli a belső figyelmeztetést, mivel a rétegek közötti lemorzsolódás csak egynél több LSTM réteg esetén alkalmazható.

Az adatok hálózaton keresztüli áramlását a *forward(self, x)* metódus vezérli. Az *out, \_ = self.lstm(x)* sor lefuttatja az időbeli sorozat-számításokat. Mivel a volatilitás-előrejelzésnél egyetlen jövőbeli pont becslése a cél.

```
class LSTMVolModel(nn.Module):
    """Kétrétegű LSTM - Kim & Won (2018), Bucci (2020)."""
    def __init__(self, input_size=1, hidden_size=64, num_layers=2, dropout=0.1):
        super().__init__()
        # PyTorch beépített LSTM réteg; batch_first=True: (batch, seq, feature) bemenet
        self.lstm = nn.LSTM(
            input_size, hidden_size,
            num_layers=num_layers,
            batch_first=True,
            # PyTorch figyelmeztetés elkerülése: 1 rétegnél a dropout paraméter hatástalan
            dropout=dropout if num_layers > 1 else 0.0
        )
        self.dropout = nn.Dropout(dropout)
        # kimeneti lineáris réteg: LSTM rejtett állapotából egyetlen volatilitás-érték
        self.fc = nn.Linear(hidden_size, 1)

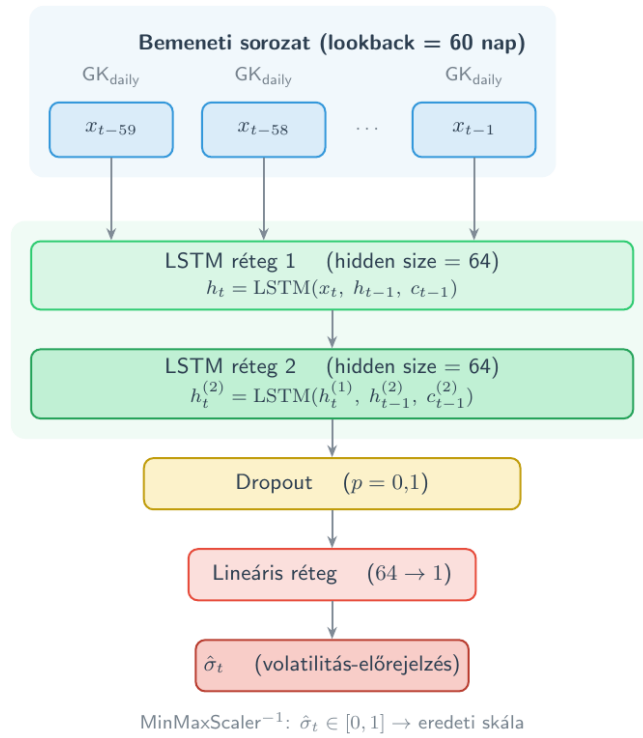
    def forward(self, x):
        out, _ = self.lstm(x)
        # csak az utolsó időlépés rejtett állapota megy a fc rétegbe
        out = self.dropout(out[:, -1, :])
        return self.fc(out).squeeze(-1)
```

4.6. ábra. A kétrétegű LSTM volatilitásmodell felépítése

#### 4.3.1. Architektúra

A kétrétegű LSTM architektúrát Bucci (2020) ajánlásai alapján konfiguráltuk. Az architektúra a 4.7. ábrán látható.

Az LSTM 60 napos visszatekintési ablakot kap bemenetként, ahol minden időlépés a napi GK-értékeket tartalmazza MinMaxScaler [0,1] normalizálással. Ez az eljárás elengedhetetlen a neurális hálózatoknál, mivel egységesíti a bemeneti változók léptékét, a szélsőértékeket is figyelembe véve. Ez a megközelítés jobb eredményt ad, mint a log-hozam alapú bemenet (Bucci, 2020). Az LSTM tanítási algoritmus és walk-forward validációja a 1. mellékletben található.



4.7. ábra. LSTM architektúra blokkváza

#### 4.3.2. Kapumechanizmus

Az LSTM modell különlegessége az úgynevezett kapumechanizmus, amely lehetővé teszi, hogy a hálózat hosszú távon szelektíven megőrizze vagy eldobja az információkat. Több kapu alkotja – mint például a felejtőkapu, amely eldönti, hogy mely információkat kell törölni. 0 és 1 közötti értékeket rendel az információkhoz, ahol 0 a teljes felejtést, 1 a teljes megőrzést jelenti. Egy másik ilyen kapu a bemeneti kapu, amely meghatározza, hogy mely új információk kerüljenek be a belső cellába. Az aktuális cellaállapot alapján a hálózat egy adott időpontbeli kimenetét a kimeneti kapu szabályozza:

$$\begin{aligned}
 f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), & (\text{felejtőkapu}) \\
 i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), & (\text{bemeneti kapu}) \\
 \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), \\
 c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & (\text{cellaállapot}) \\
 o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), & (\text{kimeneti kapu}) \\
 h_t &= o_t \odot \tanh(c_t),
 \end{aligned} \tag{7}$$

Ahol:

- $x_t$  – bemeneti vektor az adott időpontban

- $h_{t-1}$  – előző rejtett állapot
- $W_f, W_i, W_c, W_o$  – súlymátrixok
- $b_f, b_i, b_c, b_o$  – eltolási (bias) vektorok
- $\sigma$  – szigmoid függvény
- $\tanh$  – hiperbolikus tangens függvény
- $\odot$  – elemenkénti szorzás

Az LSTM hálózat kiemelkedően alkalmas a hosszú memóriájú volatilitás-folyamatok modellezésére, mivel a speciális architektúrájuknak köszönhetően képesek kezelni a hosszú távú időbeli függőségeket, amellyel a hagyományos neurális hálózatok nem boldogulnak. Ez azért kiemelkedően fontos, mivel a pénzügyi piacokon a múltbeli események hatása tartósan jelen marad, így a megbízható előrejelzésnek figyelembe kell vennie.

#### 4.3.3. Tanítás

A tanítás során az ADAM (Adaptive Moment Estimation) optimalizáló algoritmust alkalmaztuk, amely a hálózati paramétereket és a tanulási rátát automatikusan módosítja a tréning folyamán. Előnye, hogy adaptívan kezeli a gradienseket, ezzel segítve a veszteségfüggvény minimumának pontosabb megközelítését. A számítási architektúra programozási hátterét a 4.8. ábra szemlélteti. Amint a kód első sorában látható, az optimalizálót a *torch.optim.Adam* osztály példányosítja, ahol a tanulási ráta kezdeti értéke  $lr=0.001$  ( $10^{-3}$ ), amely a mély tanulási irodalomban általánosan elfogadott és robusztus alapértéknek tekinthető.

Veszteségfüggvényként az átlagos négyzetes hibát (MSE) alkalmaztuk, amely a becsült és a tényleges értékek különbségének négyzetátlagát számítja ki. Az MSE érzékenyebben reagál a nagyobb eltérésekre, így hatékonyan ösztönzi a modellt a kiugró hibák minimalizálására.

A tanítóhalmaz 15%-át validációs célra különítettük el, hogy a modell ismeretlen adatokon is pontos előrejelzést adjon.

A túlilleszkedés elkerülése érdekében early stopping technikát alkalmaztunk: a tanítás akkor áll le, ha a modell teljesítménye 5 egymást követő korszakon át nem javul (patience = 5). A modellt 21 naponként újratanítjuk.

```

# Adam optimalizáló: adaptív tanulási ráta, lr=0.001
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
# MSE veszteségfüggvény a volatilitás-előrejelzéshez
criterion = nn.MSELoss()

# korai leállítás: legjobb validációs loss állapot mentése
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_state = {k: v.clone() for k, v in model.state_dict().items()}
    wait = 0
else:
    # türelmi számláló: ha patience epoch óta nem javult, leállítás
    wait += 1
    # korai leállítás aktiválása - overfitting megelőzése
    if wait >= patience:
        break

```

4.8. ábra. Az LSTM modell tanítási beállításai és a korai leállítás

#### 4.4. LSTM-GARCH hibrid

Kim & Won (2018) nyomán egy hibrid modellt is implementáltunk, amelynek a klasszikus modellek feltételes varianciái kerülnek bemenetként:

$$\hat{\sigma}_t^{Hybrid} = LSTM(GK_{t-r:t}, \hat{\sigma}_{t-r:t}^{EWMA}, \hat{\sigma}_{t-r:t}^{GARCH}, \hat{\sigma}_{t-r:t}^{GJR}) \quad (8)$$

A hibrid megközelítés programozási háttérét és a változók összegzését a 4.9. ábra szemlélteti. Technikai szempontból az alapvető változást az *nn.Module* alapú *LSTMHybridModel* osztály inicializációja jelenti, ahol a korábbi egydimenziós bemenettel szemben egy négy bemeneti csatornával rendelkező LSTM réteget. A rejtett neuronok száma (*hidden\_size*=64), a rétegek száma (*num\_layers*=2), valamint a túltanulást gátló lemorzsolódási ráta (*dropout*=0.1) megegyezik a sima LSTM modellnél bemutatott alapbeállításokkal a közvetlen összehasonlíthatóság biztosítása érdekében.

A többváltozós bemeneti strukturálását a kód a *NumPy* könyvtár *np.column\_stack* függvényével végzi el. Ez a metódus a négy különálló modellből származó egydimenziós idősort – a Garman–Klass napi realizált volatilitást (*gk\_daily*), az exponenciálisan súlyozott mozgóátlagot (*ewma*), a klasszikus feltételes varianciát (*garch*), valamint az aszimmetrikus tőkeáttételi hatást megragadó becslést (*gjr*) az oszlopok mentén egyetlen összefüggő *raw\_features* mátrixszá fűzi össze.

A négy bemeneti jellemzőt egymástól függetlenül normalizáljuk MinMaxScaler segítségével; a többi hiperparaméter megegyezik az alap LSTM-ével. A hibrid célja, hogy a GARCH-modellek kondicionális struktúráját beépítse az LSTM memóriájába (Roszyk & Ślepaczuk, 2024).

```
# Kim & Won (2018) hibrid architektúra: input_size=4 (GK+EWMA+GARCH+GJR)
class LSTMHybridModel(nn.Module):
    """Kétrétegű LSTM hibrid - Kim & Won (2018). input_size=4."""
    # hibrid modell 4 bemeneti csatornával indul
    def __init__(self, input_size=4, hidden_size=64, num_layers=2, dropout=0.1):
        ...

    # Feature mátrix összeállítása:
    raw_features = np.column_stack([gk_daily, ewma, garch, gjr])
```

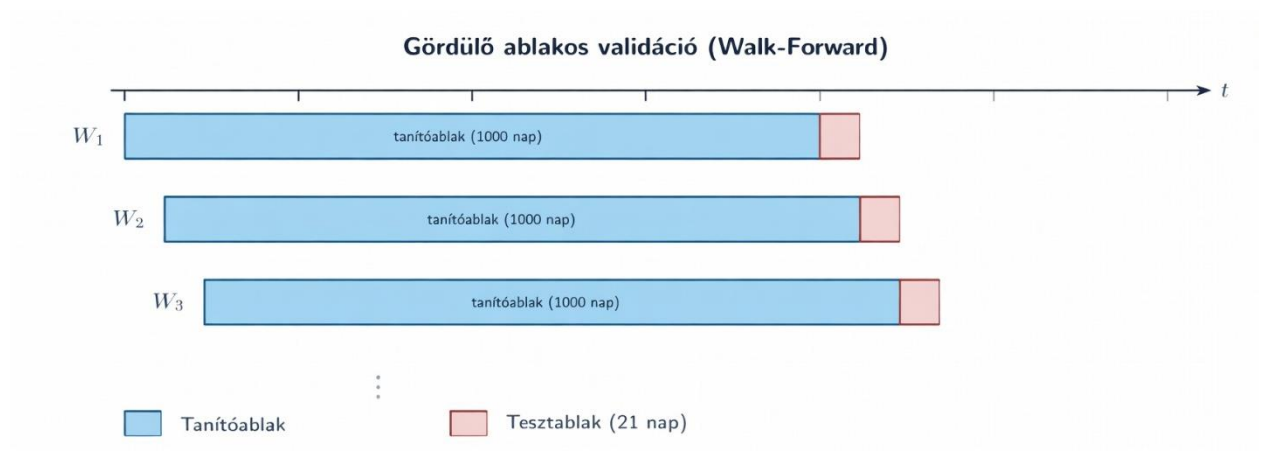
4.9. ábra. A hibrid LSTM volatilitásmodell bemeneti rétegének deklarációja

#### 4.5. Gördülő ablakos validáció

A modellek értékelése gördülő ablakos (walk-forward) validációval történik, amely valós kereskedési körülményeket szimulálja, és kizárja az előretekintési torzítást. A modellt minden  $t$  időpontban az azt megelőző 1000 kereskedési nap adataira illesztjük. A  $[t-1000, t-1]$  intervallum alkalmazása biztosítja, hogy a modell kizárólag olyan információkra támaszkodjon, amelyek a valóságban is rendelkezésre állnak. Az eljárás a 4.10. ábrán szemlélteti. A számítási háttér globális konfigurációját és a vizsgált specifikus piaci rezsimeket a 4.11. ábra forráskódja rögzíti. A modell paramétereit minden egyes ablakban az azt közvetlenül megelőző 1000 tőzsdei kereskedési nap adatára illesztjük ( $ROLLING\_WINDOWS = [1000]$ ). Az 1000 napos méret a pénzügyi matematikában standardnak tekinthető, mivel a tőzsdei munkanapok évenkénti eloszlása alapján ( $TDPY = 252$ ) ez megközelítőleg egy 4 éves historikus memóriának felel meg, ami elegendő statisztikai szabadságfokot biztosít a GARCH és LSTM alapú paraméterek konvergenciájához.

A modellek napi szinten futtatják az 1 napra előretekintő volatilitás-előrejelzést, ám a számítási kapacitások optimalizálása érdekében a modellek teljes újraillesztése diszkrét lépésközzel történik. Ezt a logikát a  $REFIT\_STEP = 21$  paraméter rögzíti, ami azt jelenti, hogy a tanító- és tesztablakok megközelítőleg havonta tolódnak előre (21 kereskedési naponta), pontosan követve a 4.10. ábrán bemutatott eltolási sémát.

A kód a `set(range(start_idx, end_idx, refit_step))` logikával generálja le azokat a diszkrét időindexeket, amikor az újraillesztés szükségessé válik. Amikor a walk-forward ciklus aktuális  $t$  időpontja eléri ezen indexek egyikét, a rendszer az összes statisztikai, ökonometriai és deep learning modellt teljesen újraszámítja és újratanítja az aktuális ablakon, elnémított diagnosztikai üzemmódban (`disp='off'`). Ez a dinamikus refit-eljárás biztosítja, hogy a modellek képesek legyenek alkalmazkodni a megváltozott piaci volatilitási rezsimekhez, miközben a mintán kívüli tesztelés során a hálózatok és együtthatók egyetlen jövőbeli adathoz sem férhetnek hozzá.



**4.10. ábra.** Gördülő ablakos validáció

```
ROLLING_WINDOWS = [1000] # 1000 napos tanítóablak
REFIT_STEP       = 21     # havonta újrabecslés

refit_indices = set(range(start_idx, end_idx, refit_step))
if t in refit_indices:
    model = arch_model(...).fit(dispatch='off')
```

**4.11. ábra.** Walk-forward mechanizmus



## 5. Értékelési keretrendszer

### 5.1. Pontossági metrikák

A pontossági metrikák olyan statisztikai mutatók, amelyek a modellek által becsült és a tényleges értékek közötti eltérést számszerűsítik. Elsődleges szerepük a modellek teljesítményének összehasonlítása és rangsorolása. Ebben a dolgozatban két metrikát használtunk a Q-LIKE veszteségfüggvényt és az RMSE átlagos négyzetes hibát.

#### 5.1.1. Q-LIKE veszteségfüggvény

Patton (2011) eredményei alapján a Q-LIKE veszteségfüggvény robusztus abban az értelemben, hogy konzisztensen rangsorolja a modelleket a valódi realizált volatilitástól különböző proxyk esetén is:

$$L_{QLIKE}(\hat{\sigma}, \sigma^*) = \frac{\sigma^{*2}}{\hat{\sigma}^2} - \ln \frac{\sigma^{*2}}{\hat{\sigma}^2} - 1, \quad (9)$$

Ahol  $\hat{\sigma}$  az előrejelzett,  $\sigma^*$  a realizált volatilitás. Kisebb érték jobb modellteljesítményt jelent. A Q-LIKE metrika algoritmikus implementációját a 5.1. ábra mutatja be. A forráskódban látható módon a realizált és az előrejelzett volatilitás-idősorokat ( $y\_true$  és  $y\_pred$ ) a rendszer először négyzetre emeli, hogy megkapja a Patton-féle képletnek megfelelő  $h\_t$  valódi és  $h\_p$  becsült variancia-idősorokat. A logaritmálás és az osztás elvégzése során a kód a nevezőkhöz és a logaritmus argumentumához egy rendkívül kicsi konstans értéket, ad hozzá ( $1e-10$ ). Ez a technikai lépés hatékonyan megakadályozza a zéróval való osztást és a negatív végtelenbe tartó logaritmusértékek, ezáltal biztosítva a teljes walk-forward folyamat algoritmikus stabilitását. Végül a `np.mean()` függvénye összesíti és átlagolja a napi diszkrét veszteségértékeket a teljes mintán kívüli időszakra vonatkozóan.

A Q-LIKE egy aszimmetrikus veszteségfüggvény, ami az alábecslést súlyosabban bünteti, mint a felülbecslést, mivel az alábecslés jelentős kockázatokkal jár, például a nem optimális befektetésekhez vezethet. Emiatt megbízhatóbb statisztikai rangsort állít fel a modellek között, mint a legtöbb veszteségfüggvény.



```
# Patton (2011):
h_t = y_true**2 # valódi variancia (GK_RV^2)
h_p = y_pred**2 # becsült variancia
qlike = np.mean(np.log(h_p + 1e-10) + h_t / (h_p + 1e-10))
```

**5.1. ábra.** Aszimmetrikus Q-LIKE veszteségfüggvény kiszámítása

### 5.1.2. RMSE

$$RMSE = \sqrt{\frac{1}{N} \sum_{t=1}^N (\hat{\sigma}_t - \sigma_t^*)^2} \quad (10)$$

Ahol  $N$  a megfigyelések száma. Az RMSE algoritmikus kiszámítását a 5.2. ábra illusztrálja. A kód a scikit-learn könyvtárból származó `mean_squared_error(y_true, y_pred)` függvénnyel először meghatározza a tényleges ( $y\_true$ ) és az előrejelzett ( $y\_pred$ ) volatilitásértékek átlagos négyzetes eltérését (MSE). Ezt követően a `np.sqrt()` függvény végrehajtja a négyzetgyökvonást, visszahelyezve a kapott hibaértéket az eredeti volatilitási dimenzióba. Az RMSE (Root Mean Squared Error) a hibatagok négyzetre emelésével nagyobb súlyt rendel a jelentős eltérésekre, ezért fokozottan érzékeny a kiugró értékekre – ezek aránytalanul nagy hatást gyakorolnak a végeredményre. Ilyen mutató alkalmazása akkor indokolt, ha a piaci sokkok és válságok pontos lekövetése a cél, mivel ilyenkor a nagy hibák büntetése kiemelten fontos a kockázatkezelés hatékonysága szempontjából.

```
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
```

**5.2. ábra.** RMSE kiszámítása

## 5.2. Statisztikai szignifikancia

A statisztikai szignifikancia egy olyan mutató, amely meghatározza, hogy egy megfigyelt eltérés megbízhatóan létezik-e, vagy csak a véletlen műve.

### 5.2.1. Diebold–Mariano teszt

Diebold-Mariano teszt nullhipotézise ( $H_0$ ) az, hogy a két vizsgált modell előrejelzési pontossága megegyezik, azaz a várható veszteségeik közötti különbség nulla. Ez azt jelenti, hogy egyik modell sem nyújt statisztikailag szignifikánsan jobb előrejelzést a másiknál:  $H_0: E(d_t) = 0$ . Az úgynevezett veszteségkülönbség (loss differential) képlete  $d_t = L(e_{1,t}) - L(e_{2,t})$ , ahol  $L$  a veszteségfüggvény és  $e_{1,t}, e_{2,t}$  a két modell hibái. Ha  $d_t < 0$ , akkor az első modell bizonyult jobbnak míg, ha  $d_t > 0$ , akkor a második modell a pontosabb.

A hipotézisvizsgálat algoritmikus felépítését a 5.3. ábra mutatja be. A forráskód kezdeti lépésként a `np.array()` struktúráját kihasználva elemenként vonja ki egymásból a két modell rögzített veszteségsorozatát, majd a `d.mean()` függvénnyel meghatározza a  $\bar{d}$  mintaátlagot.

Amennyiben a specifikált késleltetés nagyobb nullánál (*if lag > 0:*), a ciklus a  $2 * (1 - l / (lag + 1))$  Bartlett-féle lineáris súlyozási sémát alkalmazva kiszámítja a mintabeli autokovarianciákat (*acov*), és ezzel korrigálja az alapvarianciát  $var\_d = gamma0 + acov$ .

Amit valójában vizsgálunk  $DM = \frac{\bar{d}}{\sqrt{var(\bar{d})}}$ ,  $\bar{d} = \frac{1}{N} \sum_{t=1}^N d_t$  ahol  $\bar{d}$  a mintaátlag. A nullhipotézis teljesülése esetén a tesztstatisztika aszimptotikusan standard normális eloszlást ( $N(0,1)$ ) követ.

```
# veszteségkülönbség
d      = np.array(loss1) - np.array(loss2)
mean_d = d.mean()
var_d  = gamma0
    if lag > 0:
        # HAC korrekció: autokorrelált veszteségek kezelése
        acov = sum(
            2 * (1 - l/(lag+1)) *
            np.sum((d[l:] - mean_d) * (d[:-l] - mean_d)) / T
            for l in range(1, lag+1)
        )
        var_d = gamma0 + acov
dm_stat = mean_d / np.sqrt(var_d / len(d))
pval    = 2 * (1 - stats.norm.cdf(abs(dm_stat)))
```

5.3. ábra. A Diebold–Mariano tesztstatisztika és a p-érték kiszámítása

### 5.2.2. Holm–Bonferroni korrekció

A Holm-Bonferroni korrekció a többszörös tesztelés problémáját orvosolja, amely különösen fontos szerepet játszik a pénzügyi modellek teljesítményének összehasonlításakor. Ez a probléma akkor merül fel, amikor több modellt azonos adathalmazon tesztelünk. Ha  $m$  számú modellt vagy modellpárt vizsgálunk, az egyedi tesztek száma rohamosan növekszik. Megfelelő korrekció nélkül, minél több összehasonlítást végzünk, a teljes kísérletre vonatkozó elsőfajú hiba felhalmozódik. Ennek következtében drasztikusan megnő annak a valószínűsége, hogy valamelyik modellcsoport pusztán a statisztikai mintavételi véletlen vagy a piaci zaj folytán mutatkozik szignifikánsan jobbnak a többinél, hamis pozitív eredményt generálva.

Az eljárás algoritmikus megvalósítását a 5.4. ábra mutatja be. A programkód a *statsmodels.stats.multitest* modulból importálja a *multipletests* függvényt, amely képes a nyers  $p$ -értékek automatizált felülvizsgálatára. A kód a Diebold–Mariano tesztekéből származó 15 egyidejű teszt nyers, eredeti  $p$ -értékeit tömörítő vektort (*p\_raw\_values*) adja át a függvénynek, és explicit módon a *method='holm'* paraméter beállításával vezérli a korrekciós logikát.

A hagyományos, rendkívül szigorú és konzervatív Bonferroni-korrekcióval szemben, amely fixen  $m$ -mel szorozza meg az összes  $p$ -értéket, és ezáltal jelentősen rontja a teszt statisztikai erejét. Ezzel szemben a Holm-féle eljárás egy szekvenciális, lépésenként növekvő módszertant követ. Az algoritmus sorba rendezi a  $p$ -értékeket, és a legkisebb értéket terheli a legnagyobb büntetéssel, majd az index növekedésével a szorzót fokozatosan enyhíti. A függvény visszatérési értékéből a kód kinyeri a korrigált  $p$ -értékek sorozatát (*p\_korr*), miközben a felesleges kimeneteket eldobja (*\_, p\_korr, \_, \_*).

```
from statsmodels.stats.multitest import multipletests

# Holm-Bonferroni korrekció: FWER kontroll 15 egyidejű tesztnél
_, p_korr, _, _ = multipletests(p_raw_values, method='holm')
```

5.4. ábra. Bonferroni-korrekció

## 6. Eredmények

### 6.1. Előrejelzési pontosság: Q-LIKE és RMSE

A Q-LIKE és RMSE mutatókat benchmarkként alkalmazva hat modellt (EWMA, GARCH, GJR-GARCH, HAR-RV, LSTM, LSTM-GARCH) értékeltük három periódusra (Pre-COVID, COVID-sokk, kamatemelés). Az eredmények a 6.1. és 6.2. táblázatban találhatók.

Mivel ezek a mutatók a hibát mérik, minél kisebb az érték, annál jobb. A Pre-COVID időszakban az LSTM-GARCH hibrid érte el a legkisebb értéket Q-LIKE-ban (−3,701) és RMSE-ben (0,031) egyaránt. A COVID-sokk periódusában mindkét metrikában az LSTM-modell teljesített a legjobban (Q-LIKE: −2,476; RMSE: 0,060). A kamatemelési periódusban az első időszakhoz hasonlóan ismét az LSTM-GARCH bizonyult a legpontosabbnak (Q-LIKE: −2,402; RMSE: 0,037).

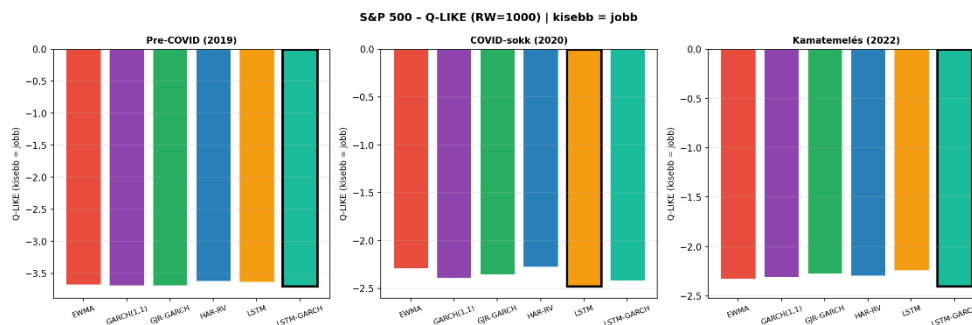
A HAR-RV abszolút pontosságban (RMSE) kiemelkedő eredményt ér el, azonban Q-LIKE-ban alulmarad. A GJR-GARCH konzervatív kockázatbecslést nyújt, ezért kockázatkezelési célokra ajánlott. Az EWMA meglepően versenyképes marad a Pre-COVID periódusban annak ellenére, hogy paraméterei nem adatból becsültek.

| Periódus    | EWMA   | GARCH  | GJR-GARCH | HAR-RV | LSTM          | LSTM-GARCH    |
|-------------|--------|--------|-----------|--------|---------------|---------------|
| Pre-COVID   | -3,672 | -3,689 | -3,690    | -3,619 | -3,636        | <b>-3,701</b> |
| COVID-sokk  | -2,286 | -2,391 | -2,353    | -2,276 | <b>-2,476</b> | -2,416        |
| Kamatemelés | -2,326 | -2,309 | -2,273    | -2,296 | -2,242        | <b>-2,402</b> |

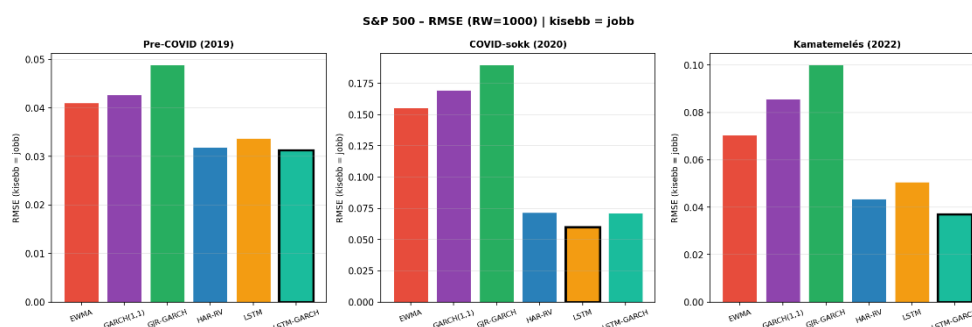
6.1. táblázat. Q-LIKE értékek periódusonként

| Periódus    | EWMA  | GARCH | GJR-GARCH | HAR-RV | LSTM         | LSTM-GARCH   |
|-------------|-------|-------|-----------|--------|--------------|--------------|
| Pre-COVID   | 0,041 | 0,043 | 0,049     | 0,032  | 0,034        | <b>0,031</b> |
| COVID-sokk  | 0,155 | 0,170 | 0,190     | 0,072  | <b>0,060</b> | 0,071        |
| Kamatemelés | 0,070 | 0,086 | 0,100     | 0,044  | 0,051        | <b>0,037</b> |

6.2. táblázat. RMSE értékek periódusonként



6.1. ábra. Q-LIKE értékek periódusonként



6.2. ábra. RMSE értékek periódusonként

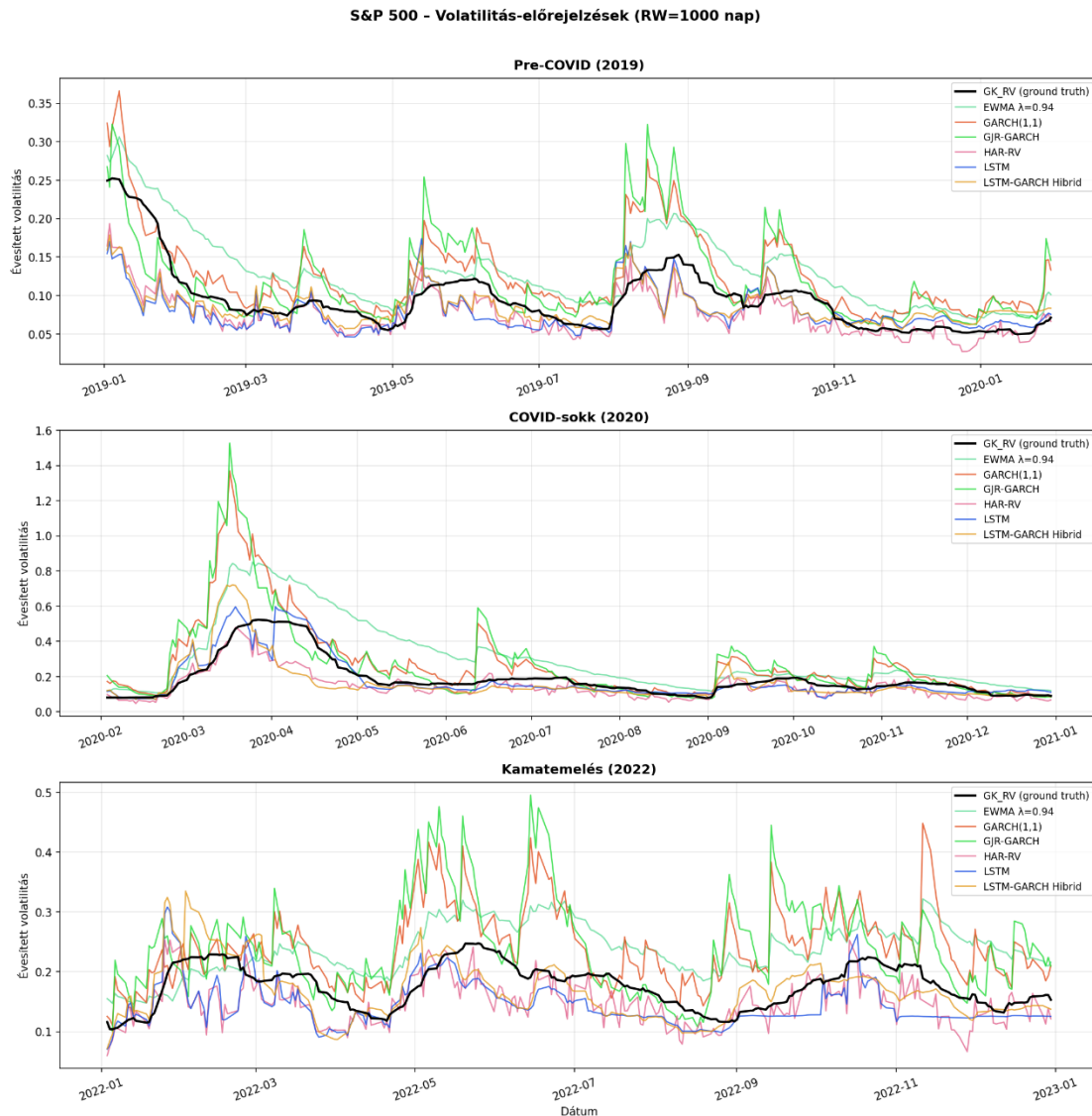
A 6.1. és 6.2. ábrán a Q-LIKE, illetve RMSE értékek vizuálisan is megerősítik a táblázatos eredményeket: a Pre-COVID és Kamatemelési periódusban az LSTM-GARCH hibrid, a COVID-sokk idején az LSTM teljesít a legjobban.

## 6.2. Volatilitás-előrejelzések vizualizációja

6.3. ábrán látható, hogy a Pre-COVID periódusban minden modell viszonylag pontosan követi a GK-RV fekete vonalát az alacsony (~8–15%) volatilitási sávban. Az EWMA és a GJR-GARCH a lokális csúcsoknál túlreagálnak (felülbecsülnek), ezzel szemben a HAR-RV és az LSTM-GARCH hibrid a legpontosabban követi a benchmarkot.

A COVID-sokk periódusában óriási szétterülés jellemző a modellek között. A klasszikus modellek (GARCH, GJR-GARCH) jelentősen alábecsülik a 2020 márciusi csúcsot, majd lassan alkalmazkodnak. Az LSTM és az LSTM-GARCH hibrid reaktívabban követi a sokkokat, szintén némi késéssel – ez a gördülő ablak és az újraillesztési frekvencia (21 nap) korlátja.

A kamatemelési periódusban a modellek már nem terülnek annyira szét, mint a COVID-sokk idején. Tartós eltérésként megfigyelhető, hogy a GJR-GARCH és az EWMA következetesen felülbecsüli a volatilitást, míg a HAR-RV és az LSTM-GARCH egyes időszakokban alulmarad a realizált értékhez képest.



**6.3. ábra.** Volatilitás előrejelzések

### 6.3. Diebold–Mariano tesztek

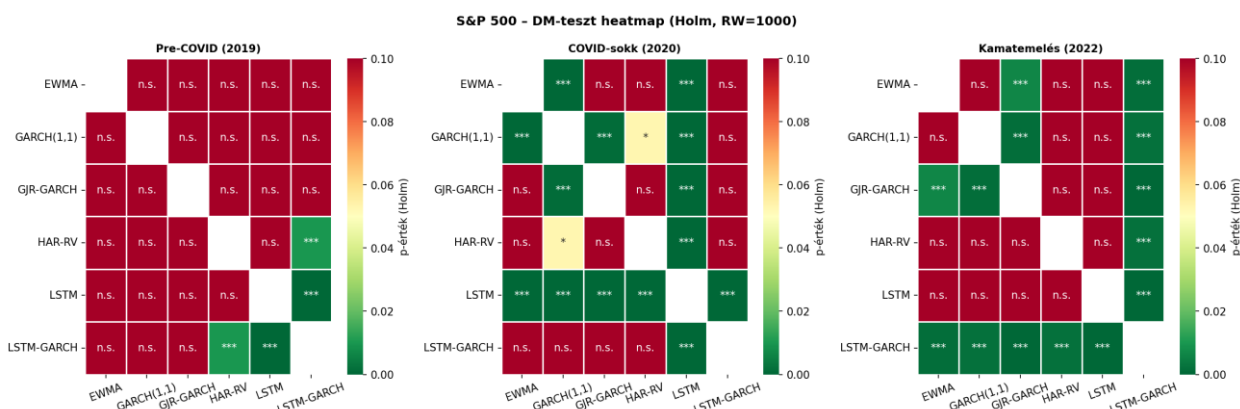
A heatmap sorai és oszlopai a modelleket jelölik. Az egyes cellák a sor- és oszlopmodell közötti DM-teszt szignifikanciáját mutatják (zöld = a sormodell szignifikánsan jobb, piros = rosszabb). A szignifikanciaszintek értelmezése:  $p < 0,01$  esetén rendkívül szignifikáns – a nullhipotézis 99%-os megbízhatósággal elutasítható;  $p < 0,05$  esetén szignifikáns, a különbség 95%-os valószínűséggel nem a véletlen műve;  $p < 0,10$  esetén csak 90%-os szinten mutatható ki eltérés; n.s. (not significant) jelölés esetén nincs statisztikailag kimutatható különbség a két modell között. Az eredmények a 6.4. ábrán láthatók.

A kamatemelési periódusban az LSTM-GARCH heatmapje a legzöldebb: a hibrid modell minden klasszikus modellt szignifikánsan felülmúl.

A Pre-COVID periódusban a szignifikancia gyengébb, sok n.s. cella látható, ami arra utal, hogy a modellek hasonlóan teljesítenek. Az LSTM-GARCH szignifikánsan jobb az EWMA-nál és a GARCH-nál, mivel a hibrid modell egyszerre képes megragadni a volatilitás klasszikus dinamikáját és a nemlineáris mintázatokat. A HAR-RV-vel szemben azonban nem mutatható ki szignifikáns különbség.

A COVID-sokk idején az LSTM szignifikánsan felülmúlja az összes hagyományos modellt. Az ábrán is jól látható, hogy válságidőszakban a modellek teljesítménye mennyire erősen szétválik.

A Holm-korrekciónak hatására 12 teszt szignifikanciaszintje módosult; korrekció nélkül több hamis pozitív eredmény jelent volna meg.



6.4. ábra. DM hőtérkép

## 6.4. Rezsimfüggő teljesítmény

Az alábbiakban a három periódus eredményeit foglaljuk össze, kiemelve az egyes piaci körülmények között mutatkozó teljesítménykülönbségeket. A stabil Pre-COVID rezsimben az EWMA Q-LIKE mutatóban meglepően versenyképes maradt, ami a  $\lambda=0,94$ -es reaktivitási előnnyel magyarázható: a modell gyorsan követi a volatilitás fokozatos változásait, anélkül, hogy túlreagálna az egyszeri sokkokra. A COVID-sokk idején a klasszikus modellek közül a GARCH Q-LIKE mutatóban elért eredménye a feltételes varianciastruktúrájának köszönhető: az extrém sokk után a modell a hosszú távú átlaghoz való visszatéréssel lassítja az alul- és felülbecslések váltakozását, ami a Q-LIKE aszimmetrikus súlyozásánál előnyös. A kamatemelési periódusban az alacsony ACF-struktúra miatt a modellek közel azonos

pontossággal teljesítenek, és a különbségek statisztikailag gyengébbek. A 6.3. táblázat összefoglalja, hogy melyik modell bizonyult a legalkalmasabbnak az egyes időszakokban.

| <b>Periódus</b> | <b>Legjobb volatilitás-modell (Q-LIKE)</b> | <b>Jellemző piaci dinamika</b>             |
|-----------------|--------------------------------------------|--------------------------------------------|
| Pre-COVID       | LSTM-GARCH                                 | Alacsony volatilitás, magas autokorreláció |
| COVID-sokk      | LSTM                                       | Extrém aszimmetria                         |
| Kamatemelés     | LSTM-GARCH                                 | Magasabb zaj, alacsonyabb autokorreláció   |

**6.3. táblázat.** Legalkalmasabb modellek



## 7. Következtetések

Az eredmények alapján a legjobban teljesítő modell az LSTM-GARCH hibrid, amely ötvözi a modern neurális hálózatot a klasszikus ökonometriai modellel, megőrizve mindkét megközelítés legjobb tulajdonságait. Ugyanakkor a volatilitás-előrejelzésben nincs egyetlen domináns modell: a teljesítmény erősen rezsimfüggő. Stabil piaci környezetben (Pre-COVID) az LSTM-GARCH hibrid bizonyult a legpontosabbnak, míg az extrém stresszperiódusban (COVID-sokk) az önálló LSTM teljesített a legjobban mind RMSE-ben, mind Q-LIKE-ban. A HAR-RV abszolút pontosságban (RMSE) kiemelkedő, azonban Q-LIKE-ban alulmarad – ez rámutat arra, hogy eltérő veszteségfüggvények eltérő modellrangsorokat eredményezhetnek. A Diebold–Mariano-tesztek alapján a modellek közötti teljesítménykülönbségek statisztikailag szignifikánsak, különösen a kamatemelési periódusban, ami jelzi, hogy az eltérések nem véletlenszerűek.

A kutatásnak ugyanakkor vannak korlátai: kizárólag az S&P 500 indexet vizsgáltuk, más tőzsdei indexeken eltérő modellek bizonyulhatnak dominánsnak. Az előre meghatározott hiperparaméterek és a 21 naponkénti újraillesztés szintén befolyásolhatja az optimalizálást. A jövőbeli kutatások lehetséges irányai közé tartozik több tőzsdei index párhuzamos vizsgálata, Transformer architektúrák alkalmazása, magasabb frekvenciájú adatok bevonása, valamint adaptív újraillesztési stratégiák kidolgozása.

## Irodalomjegyzék

- Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.
- Bucci, A. (2020). Realized volatility forecasting with neural networks. *Journal of Financial Econometrics*, 18(3), 502–531.
- Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2), 174–196.
- Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263.
- Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.
- Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
- Garman, M. B., & Klass, M. J. (1980). On the estimation of security price volatilities from historical data. *Journal of Business*, 53(1), 67–78.
- Gál, H. (2022). A devizaárfolyamok és a részvénytőkepiac volatilitásának előrejelzése a GARCH modellekben. *E-CONOM*, 11(1), 42–48.
- Glosten, L. R., Jagannathan, R., & Runkle, D. E. (1993). On the relation between the expected value and the volatility of the nominal excess return on stocks. *Journal of Finance*, 48(5), 1779–1801.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Kim, H. Y., & Won, C. H. (2018). Forecasting the volatility of stock price index: A hybrid model integrating LSTM with multiple GARCH-type models. *Expert Systems with Applications*, 103, 25–37.
- Nelson, D. B. (1991). Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, 59(2), 347–370.
- Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1), 246–256.

- Poon, S. H., & Granger, C. W. J. (2003). Forecasting volatility in financial markets: A review. *Journal of Economic Literature*, 41(2), 478–539.
- RiskMetrics Group (1996). RiskMetrics Technical Document (4th ed.). J.P. Morgan/Reuters.
- Romano, J. P., & Wolf, M. (2005). Stepwise multiple testing as formalized data snooping. *Econometrica*, 73(4), 1237–1282.
- Roszyk, P., & Ślepaczuk, R. (2024). Hybrid LSTM-GARCH models for volatility forecasting. Working Paper, University of Warsaw.

## Mellékletá

### 1. melléklet: Felhasznált csomagok

| Csomag              | Felhasználás                      |
|---------------------|-----------------------------------|
| yfinance            | Adatbetöltés (Yahoo Finance)      |
| pandas              | Adatkezelés                       |
| numpy               | Numerikus számítás                |
| arch                | GARCH, GJR-GARCH illesztés        |
| statsmodels         | HAR-RV, ACF/PACF, Holm-Bonferroni |
| torch               | LSTM neurális háló                |
| sklearn             | MinMaxScaler, RMSE                |
| matplotlib/ seaborn | Vizualizáció                      |
| scipy               | Student-illesztés                 |